==========

==========

CHAPTER 20 — ADVANCED VIRTUALIZATION TECHNIQUES

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of advanced ARMv8 virtualization mechanisms, nested virtualization, shadow page tables, paravirtualization, memory overcommit, live migration, and hypervisor-level optimizations.

****

****

20.1 — Introduction to Advanced Virtualization

****

****

ARMv8 virtualization is built on:

- **Stage-1 translation** (guest OS)
- **Stage-2 translation** (hypervisor)
- **VMID-tagged TLB entries**
- **HCR_EL2 virtualization controls**
- **IPA → PA mapping**
- **trap-and-emulate mechanisms**

Advanced virtualization techniques extend these foundations to support:

- nested virtualization
- memory overcommit
- live migration
- shadow page tables
- paravirtualization
- device virtualization
- hypervisor optimizations

This chapter explains these techniques in maximum detail.

****

****

20.2 — Nested Virtualization

****

****

Nested virtualization allows:

- a guest hypervisor (L1)
- to run guest VMs (L2)
- inside a real hypervisor (L0)

ARMv8 does not natively support nested virtualization. It must be **emulated** by the L0 hypervisor.

Nested Virtualization Components

- **Shadow Stage-2 tables**
- **Shadow VMIDs**
- **Shadow TLB invalidation**
- **Trap-and-emulate for HCR_EL2**
- **Trap-and-emulate for Stage-2 faults**
- **Nested BBM sequences**
- **Nested TLBI sequences**

Performance Impact

Nested virtualization is expensive because:

- every Stage-2 update by L1 must be emulated
- every TLBI by L1 must be emulated
- every descriptor update must be shadowed

****

****

20.3 — Shadow Page Tables

****

****

Shadow page tables are used when:

- guest hypervisor (L1) manages Stage-2 tables
- real hypervisor (L0) must maintain control

Shadowing Process

1. L1 writes Stage-2 descriptor
2. Trap to L0
3. L0 validates descriptor
4. L0 writes descriptor into shadow Stage-2 table
5. L0 updates real Stage-2 table
6. L0 performs TLBI
7. L0 resumes L1

Shadow Tables Required For

- Stage-2 mappings
- Stage-2 permissions
- Stage-2 attributes
- Stage-2 shareability

Shadow Table Challenges

- maintaining consistency
- handling BBM
- handling TLBI
- handling walk cache invalidation

****

****

20.4 — Paravirtualization

****

****

Paravirtualization improves performance by:

- reducing traps
- reducing emulation
- reducing shadowing
- reducing TLBI overhead

Guest OS or guest hypervisor uses **hypercalls** instead of:

- modifying Stage-2 tables directly
- performing TLBI directly
- performing BBM directly

Benefits

- fewer traps
- fewer shadow updates
- fewer TLB invalidations
- higher performance

Examples

- paravirtualized MMU updates
- paravirtualized DMA
- paravirtualized interrupts
- paravirtualized timers

****

****

20.5 — Memory Overcommit

****

****

Memory overcommit allows:

- more guest memory allocated
- than physical memory available

Hypervisor uses:

- ballooning
- swapping
- compression
- deduplication
- lazy allocation

Ballooning

Guest OS inflates balloon driver → frees memory. Hypervisor deflates balloon → allocates memory.

Swapping

Hypervisor swaps guest pages to disk.

Compression

Hypervisor compresses cold pages.

Deduplication

Hypervisor merges identical pages.

Lazy Allocation

Hypervisor allocates pages only when accessed.

****

****

20.6 — Live Migration

****

****

Live migration moves a running VM between hosts.

Requirements

- Stage-2 remapping
- dirty page tracking
- memory copying
- CPU state copying
- device state copying
- network state copying

Dirty Page Tracking

Hypervisor uses:

- S2AP
- AF

- write-protection
- traps

Migration Steps

1. Pre-copy memory
2. Track dirty pages
3. Copy dirty pages
4. Freeze VM
5. Copy CPU/device state
6. Resume VM on destination

****

****

20.7 — Device Virtualization

****

****

Device virtualization allows VMs to use hardware devices safely.

Techniques

- trap-and-emulate
- MMIO virtualization
- interrupt virtualization
- DMA virtualization
- PCIe virtualization
- IOMMU integration

MMIO Virtualization

Hypervisor intercepts:

- MMIO reads
- MMIO writes

Hypervisor emulates device behavior.

Interrupt Virtualization

Hypervisor controls:

- GIC virtualization
- interrupt routing
- interrupt injection

****

****

20.8 — IOMMU Integration

****

****

IOMMU provides:

- DMA isolation
- device memory protection
- device virtualization

IOMMU performs:

```
Device DMA → IOVA → PA
```

Hypervisor controls:

- IOVA → PA mappings
- device permissions
- device attributes

Benefits

- prevents DMA attacks
- isolates VMs
- supports SR-IOV

****

****

20.9 — Stage-2 Optimization Techniques

****

****

Stage-2 is expensive. Optimizing it improves virtualization performance.

Best Practices

- use block mappings
- minimize fragmentation
- minimize TLBI
- batch TLBI
- batch descriptor updates
- use WB cacheability
- use Inner Shareable

Avoid

- excessive page mappings
- excessive TLBI
- excessive BBM
- excessive shadowing

****

****

20.10 — Hypervisor Scheduling

****

****

Hypervisor scheduling determines:

- VM fairness
- VM latency
- VM throughput
- interrupt latency
- device access latency

Techniques

- credit scheduling
- priority scheduling
- partitioned scheduling
- real-time scheduling

QNX/RTOS

QNX uses:

- deterministic scheduling
- partitioned scheduling
- strict priority rules

****

****

20.11 — Hypervisor Security

****

****

Hypervisor enforces:

- VM isolation
- memory protection
- device protection
- DMA protection
- interrupt protection

Techniques

- Stage-2 permissions
- Stage-2 attributes
- S2PXN/S2UXN
- VMID isolation
- IOMMU isolation
- trap-and-emulate

****

****

20.12 — Hypervisor Debugging

****

****

Hypervisor debugging requires:

- Stage-2 fault analysis
- HPFAR_EL2 inspection
- ESR_EL2 inspection
- shadow table inspection
- VMID tracking
- TLB tracking

Tools

- page table dumpers
- TLB dumpers
- VMID trackers
- shadow table analyzers

****

****

20.13 — Hypervisor Performance Counters

****

****

Hypervisor uses PMU counters to track:

- TLB misses
- page-table walks
- Stage-2 faults
- cache misses
- memory bandwidth
- device latency

Optimization

- reduce TLB misses
- reduce Stage-2 faults
- reduce fragmentation

****

****

20.14 — QNX/RTOS Virtualization

****

****

QNX uses virtualization for:

- safety partitions
- deterministic isolation
- device partitioning
- memory partitioning

QNX Techniques

- static VMID ranges
- static ASID ranges
- deterministic Stage-2 mappings
- deterministic device mappings

****

****

20.15 — Full Advanced Virtualization Example

****

****

Scenario

Guest hypervisor (L1) updates Stage-2 descriptor.

Steps

1. L1 writes descriptor
2. Trap to L0
3. L0 validates descriptor
4. L0 updates shadow table
5. L0 updates real Stage-2 table
6. L0 performs TLBI
7. L0 resumes L1

Result

Nested virtualization works correctly.

****

****

20.16 — Chapter Summary

****

****

This chapter covered:

- nested virtualization
- shadow page tables
- paravirtualization
- memory overcommit
- live migration
- device virtualization
- IOMMU integration
- Stage-2 optimization
- hypervisor scheduling
- hypervisor security
- hypervisor debugging
- hypervisor performance counters
- QNX/RTOS virtualization
- full examples